

Insecure Direct Object Reference (IDOR) – Exploitation with Burp Suite and ffuf

1. Overview of IDOR

Insecure Direct Object Reference (IDOR) is an access control vulnerability that occurs when an application exposes a direct reference to an internal resource (e.g., a database record, file, or object) through a user-controllable parameter, without proper authorization checks.

Common examples:

- https://insecure-website.com/customer_account?customer_number=132355 – changing the customer_number may reveal other users' data.
- Static file references: https://site.com/transcripts/transcript_123.txt – incrementing the filename may expose other transcripts.

Impacts:

- **Horizontal privilege escalation:** View or modify data belonging to other users with the same role.
- **Vertical privilege escalation:** Access data or functions intended for higher-privileged users (e.g., admin accounts).
- **Data leakage:** Exposure of sensitive personal information, credentials, or internal files.

2. Lab Environment – IDOR 0x01

The lab (Labs / IDOR 0x01) presents a page that displays account details based on a numeric account parameter in the URL.

Initial legitimate access (as a logged-in user) shows:

text

Your account details

Username: kate

Address: 131, Duchess Drive, LDN 3AW

Type: user

The URL pattern is:

text

<http://localhost/labs/e0x02.php?account=<number>>

The parameter account appears to be a direct reference to a user record in the database.

3. Manual Testing – Changing the Parameter

We can manually modify the account number in the browser address bar.

Example attempts (from screenshots):

- ?account=1010 → displays:

text

Username: Jack

Address: 121, Sparrow Street, PRT 4BD

Type: admin

This is a **vertical privilege escalation** – we accessed an admin account.

- ?account=1016 → displays:

text

Username: peter

Address: 181, Parker Place, USA 4PP

Type: user

Thus, the parameter is directly tied to user IDs, and no authorization check prevents us from viewing other users.

4. Automated Enumeration with ffuf

To discover all accessible accounts, we can fuzz the account parameter with a list of numbers. The screenshots show the use of ffuf (Fuzz Faster U Fool) – a web fuzzing tool written in Go.

4.1 Generating the Wordlist

We need a list of numbers from 1 to 2000 (as seen in the lab). Use Python to generate it:

bash

```
python3 -c 'for i in range(1, 2001): print(i)' > number.txt
```

Verify:

bash

```
head number.txt
```

Output:

text

1

2

3

...

Tail:

bash

tail number.txt

text

1991

1992

...

2000

4.2 Fuzzing with ffuf

Command:

bash

```
ffuf -u 'http://localhost/labs/e0x02.php?account=FUZZ' -w number.txt -fs 849
```

Explanation:

- -u – target URL with FUZZ as the placeholder.
- -w – wordlist file (number.txt).
- -fs 849 – **filter responses by size**. The normal responses (for non-admin users) have a size of 849 bytes (as seen in the initial pages). By filtering out that size, we only see responses with different sizes (e.g., admin account may have more or less content, or a different status code).

Output (from screenshots):

text

1978 [Status: 200, Size: 849, Words: 189, Lines: 33, Duration: 4819ms]

1971 [Status: 200, Size: 849, Words: 189, Lines: 33, Duration: 4821ms]

1970 [Status: 200, Size: 849, Words: 189, Lines: 33, Duration: 4821ms]

:: Progress: [2000/2000] :: Job [1/1] :: 41 req/sec :: Duration: [0:00:05] :: Errors: 0 ::

All those responses had size 849 (filtered out), so they are not shown in the final output (the user piped to grep FUZZ to see only interesting ones). After fuzzing, the interesting accounts (with different sizes) were revealed.

To capture only unique responses, we can also use -fs 849 to hide common ones, or use -fc to filter status codes.

5. Using Burp Suite for IDOR

Instead of command-line fuzzing, we can use **Burp Intruder** in a similar way:

1. **Intercept** the request for the account page (e.g., GET /labs/e0x02.php?account=1010).
 2. **Send to Intruder** (Ctrl+I).
 3. In the **Positions** tab, clear existing markers and highlight the numeric value of account, click **Add \$**.
 4. In the **Payloads** tab, load the number.txt wordlist (or generate a numeric payload using the **Numbers** payload type).
 5. Start the attack and sort the results by **Length** or **Status** to identify accounts with different response sizes or content.
 6. Look for responses containing admin in the body.
-

6. Findings

The fuzzing revealed at least two significant accounts:

Account ID	Username	Address	Role
1010	Jack	121, Sparrow Street, PRT 4BD	admin
1016	peter	181, Parker Place, USA 4PP	user

Additionally, the default account for kate (not shown in fuzzing but known from the lab) likely has ID 1 or similar.

Vertical privilege escalation was achieved by accessing the admin account (ID 1010), bypassing the intended access control.

7. Step-by-Step Exploitation Summary

Step	Action
------	--------

- | | |
|----|---|
| 1. | Identify the direct object reference in the URL: account parameter. |
| 2. | Manually change the parameter to test other numeric values (e.g., 1010). Observe that you can view other users' details. |
| 3. | Generate a list of possible IDs (e.g., 1–2000). |
| 4. | Use ffuf (or Burp Intruder) to fuzz the parameter, filtering out responses with the default size (849 bytes) to focus on anomalies. |
| 5. | Analyse the results – find accounts with different sizes or containing keywords like admin. |
| 6. | Access the admin account directly with the discovered ID. |

8. Prevention of IDOR

- **Implement proper access control checks** on the server side – verify that the authenticated user has permission to access the requested object.
- **Use indirect references** – map internal IDs to random, unpredictable tokens (e.g., UUIDs) and store the mapping server-side.
- **Avoid exposing internal identifiers** in URLs or client-side parameters.
- **Validate and authorise** every request that references a resource.

9. Conclusion

The lab demonstrates a classic IDOR vulnerability where the account parameter directly maps to database records without any authorisation. By brute-forcing the parameter, we were able to enumerate user accounts and escalate privileges to an admin account. This highlights the importance of robust access controls and the dangers of exposing direct object references.